Environment Variables

Now that we have our feet under us when it comes to using the Command Prompt let us discuss one of the more critical topics when thinking about how applications and scripting work in Windows, `Environment Variables`. In this section, we will discuss what they are, their uses, and how we can manage the variables in our system.

What an Environment Variable Is

Environment variables are settings that are often applied globally to our hosts. They can be found on Windows, Linux, and macOS hosts. This concept is not specific to one OS type, but they function differently on each OS. Environment variables can be accessed by most users and applications on the host and are used to run scripts and speed up how applications function and reference data. On a Windows host, environment variables are `not` case sensitive and can have spaces and numbers in the name. The only real catch we will find is that they cannot have a name that starts with a number or include an equal sign. When referenced, we will see these variables called like so:

Environment Variables

```
%SUPER_IMPORTANT_VARIABLE%
```

It is normal to see these variables (especially those already built into the system) displayed in uppercase letters and utilizing an underscore to link any words in the name. Before moving on, we should mention one crucial concept regarding environment variables known as `Scope`.

Variable Scope

In this context, `Scope` is a programming concept that refers to where variables can be accessed or referenced. 'Scope' can be broadly separated into two categories:

- `Global:`
  - `Global variables are accessible globally. In this context, the global scope lets us know that we can access and reference the data stored inside the variable from anywhere within a program.`
- `Local:`
  - `Local variables are only accessible within a local context. Local means that the data stored within these variables can only be accessed and referenced within the function or context in which it has been declared.`

Let us walk through an example scenario together to understand the differences better. In this scenario, we have two users, `Alice` and `Bob`. Both users have a default command prompt session and are logged in concurrently to the same machine. Additionally, both users issue a command to print out the data stored within the `%WINDIR%` variable, as seen in the examples below.

Showcasing Global Variables

Example 1:

Environment Variables

Integrated Terminal

```
┌─[us-academy-1]─[10.10.14.155]─[htb-ac-819475@htb-byk9ife5jk]─[~]
└──╼ [★]$
```

Example 2:

Environment Variables

```
C:\Users\bob> echo %WINDIR%

C:\Windows
```

We can see that this variable is accessible to both users. As such, both users can display the data stored within it. This is because the `%WINDIR%` variable is a `global variable` as defined by the Windows OS. However, what if Alice wanted to create a secret variable that Bob could not view or access; how would she go about doing so?

Showcasing Local Variables

Example 1:

Environment Variables

```
C:\Users\alice> set SECRET=HTB{5UP3r_53Cr37_V4r14813}

C:\Users\alice> echo %SECRET%
HTB{5UP3r_53Cr37_V4r14813}
```

Example 2:

Environment Variables

```
C:\Users\bob> echo %SECRET%
%SECRET%

C:\Users\bob> set %SECRET%
Environment variable %SECRET% not defined
```

In the first example, Alice creates a variable called `SECRET` and stores the value `HTB{5UP3r_53Cr37_V4r14813}` inside it. After setting the value of the variable, Alice then retrieves it using the command `echo` to print out the value stored within. However, when Bob attempts to retrieve the same variable, he cannot, as it is not defined in his current environment. What Alice created was a `local variable` that only she could access as it was only defined in the context of her local environment.

Note: This explanation of global vs. local scope is in no way a fully comprehensive guide to the differences between them and will not include more advanced concepts. This section is to provide the necessary background information moving forward.

Now that we have a basic understanding of variables and a general idea of the basics behind defined `scopes`, let us dig into how Windows interacts and stores environment variables and how we can interact with them. Like before, Windows, like any other program, contains its own set of variables known as `Environment Variables`. These variables can be separated into their defined scopes known as `System` and `User` scopes. Additionally, there is one more defined scope known as the `Process` scope; however, it is volatile by nature and is considered to be a sub-scope of both the `System` and `User` scopes. Keeping this in mind, let's explore their differences and intended functionalities.

Integrated Terminal

Scope	Description	Required to Access	Registry Location
`System (Machine)`	The System scope contains environment variables defined by the Operating System (OS) and are accessible globally by all users and accounts that log on to the system. The OS requires these variables to function properly and are loaded upon runtime.	Local Administrator or Domain Administrator	`HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Environment`
`User`	The User scope contains environment variables defined by the currently active user and are only accessible to them, not other users who can log on to the same system.	Current Active User, Local Administrator, or Domain Administrator	`HKEY_CURRENT_USER\Environment`
`Process`	The Process scope contains environment variables that are defined and accessible in the context of the currently running process. Due to their transient nature, their lifetime only lasts for the currently running process in which they were initially defined. They also inherit variables from the System/User Scopes and the parent process that spawns it (only if it is a child process).	Current Child Process, Parent Process, or Current Active User	`None (Stored in Process Memory)`

The table should provide good overall coverage of how Windows deals with environment variables and how only certain users can access certain variables due to permissions. Now that we understand these differences, let us begin attempting to make specific changes to environment variables ourselves.

Using Set and Echo to View Variables

To understand the changes to environment variables taking place, we need some way to view their contents via the command prompt. Thankfully, we have a couple of choices available to us: `set` and `echo`.

Display with Set

Environment Variables

```
C:\Users\htb\Desktop>set %SYSTEMROOT%

Environment variable C:\Windows not defined
```

Upon opening the command prompt, you can issue the command `set` to print all available environment variables on the system. Alternatively, you can enter the same command again with the variable's name without setting it equal to anything to print the value of a specific variable. We see that in this case, it mentions the value itself is not defined; however, this is because we are not defining the value of `%SYSTEMROOT%` using `set` in this example.

Display with Echo

Integrated Terminal

```
C:\Users\htb\>echo %PATH%

C:\Users\htb\Desktop
```

Similar to the example above, you can use `echo` to display the value of an environment variable. Unlike the previous command, `echo` is used to print the value contained within the variable and has no additional built-in features to edit environment variables. In the next section, we will discuss how we create new variables, remove unneeded ones, and edit existing ones using the command prompt.

Managing Environment Variables

Now that we have some way to view existing environment variables on our system, we need to be able to create, remove, and manage them from the comfort and safety of our prompt. We have two methods available to us to do so. We can either use `set` or `setx` to perform our intended actions.

When to Use `set` Vs. `setx`

Both set and setx are command line utilities that allow us to display, set, and remove environment variables. The difference lies in how they achieve those goals. The `set` utility only manipulates environment variables in the current command line session. This means that once we close our current session, any additions, removals, or changes will not be reflected the next time we open a command prompt. Suppose we need to make permanent changes to environment variables. In that case, we can use `setx` to make the appropriate changes to the registry, which will exist upon restart of our current command prompt session.

Note: Using `setx`, we also have some additional functionality added in, such as being able to create and tweak variables across computers in the domain as well as our local machine.

We should now be familiar with some of the primary differences between both commands discussed above. There will be times and situations when one should be prioritized over the other. As an attacker, there will be times when we will need to enumerate existing environment variables for information. Let us move on and create some actual variables we can use.

Creating Variables

Creating environment variables is quite a simple task. We can use either `set` or `setx` depending on the task at hand and our overall goal. The following examples will show both being put into action to give us a feel for the syntax surrounding either command. Please note that the syntax between both in some cases is very similar; however, `setx` does have some additional features that we will attempt to explore here. Additionally, to ensure things are not getting too repetitive, we will only show both the `set` and `setx` commands for creating variables and utilizing `setx` for every other example. Just know that the syntax between creating, removing, and editing environment variables is identical.

Let us go ahead and create a variable to hold the value of the IP address of the Domain Controller (`DC`) since we might find it useful for testing connectivity to the domain or querying for updates. We can do this using the `set` command.

Using set

Environment Variables

```
C:\htb> set DCIP=172.16.5.2
```

Upon running this command, there is no immediate output. However, know that the variable has been set for our current command

Integrated Terminal

Validating the Change

Environment Variables

```
C:\htb> echo %DCIP%

172.16.5.2
```

As we can see, the environment variable `%DCIP%` is now set and available for us to access. As stated above, this change is considered part of the `process` scope, as whenever we exit the command prompt and start a new session, this variable will cease to exist on the system. We can remedy this situation by permanently setting this variable in the environment using `setx`.

Using setx

Environment Variables

```
C:\htb> setx DCIP 172.16.5.2

SUCCESS: Specified value was saved.
```

From this example, we can see that the syntax between commands varies slightly. Previously, we had to set the variable's value equal to the variable itself. Here we have to provide the variable's name followed by the value. The syntax is as follows: `setx <variable name> <value> <parameters>`. After running this command, we see that our value was saved in the registry since we were provided with the `SUCCESS` message. Of course, if we are curious if the value is truly set, we can validate it exactly as done above. Remember that this change will only occur after we open up another command prompt session. On a remote system, variables created or modified by this tool will be available at the next logon session.

Editing Variables

In addition to creating our own variables, we can edit existing ones. Since we are already familiar with creating them, editing is just as easy, except we will replace the existing values. Let us say that the IP address of our `DC` changed, and we need to update the value of our custom environment variable to reflect this change.

Using setx

Environment Variables

```
C:\htb> setx DCIP 172.16.5.5

SUCCESS: Specified value was saved.
```

In the previous example, we set `172.16.5.2` as the value for the DC on the network; however, using `set`, we can update this value by simply setting the value again to our new address, `172.16.5.5`.

Validating the edit

Environment Variables

```
C:\htb> echo %DCIP%

172.16.5.5
```

Integrated Terminal

We have successfully edited our initial custom variable to reflect the DC's IP change. We can now move on and discuss removing variables.

Removing Variables

Much like creating and editing variables, we can also remove environment variables in a very similar manner. To remove variables, we cannot directly delete them like we would a file or directory; instead, we must clear their values by setting them equal to nothing. This action will effectively delete the variable and prevent it from being used as intended due to the value being removed. In our first example, we created the variable `%DCIP%` containing the value of the IP address of the domain controller on the network and permanently saved it into the registry. We can attempt to remove it by doing the following:

Using setx

Environment Variables

```
C:\htb> setx DCIP ""


SUCCESS: Specified value was saved.
```

This command will remove `%DCIP%` from our system's current environment variables and will also be reflected in the registry once we open a new command prompt session. We can verify that this is indeed the case by doing the following:

Verifying the Variable has Been Removed

Environment Variables

```
C:\htb> set DCIP
Environment variable DCIP not defined

C:\htb> echo %DCIP%
%DCIP%
```

Using both `set` and `echo`, we can verify that the `%DCIP%` variable is no longer set and is not defined in our environment anymore.

Important Environment Variables

Now that we are comfortable creating, editing, and removing our own environment variables, let us discuss some crucial variables we should be aware of when performing enumeration on a host's environment. Remember that all information found here is provided to us in clear text due to the nature of environment variables. As an attacker, this can provide us with a wealth of information about the current system and the user account accessing it.

Variable Name	Description
`%PATH%`	Specifies a set of directories(locations) where executable programs are located.
`%OS%`	The current operating system on the user's workstation.
`%SYSTEMROOT%`	Expands to `C:\Windows`. A system-defined read-only variable containing the Windows system folder. Anything Windows considers important to its core functionality is found here, including important data, core system binaries, and configuration files.
`%LOGONSERVER%`	Provides us with the login server for the currently active user followed by the machine's hostname. We can use this information to know if a machine is joined to a domain or workgroup.
`%USERPROFILE%`	Provides us with the location of the currently active user's home directory. Expands to `C:\Users\{username}`

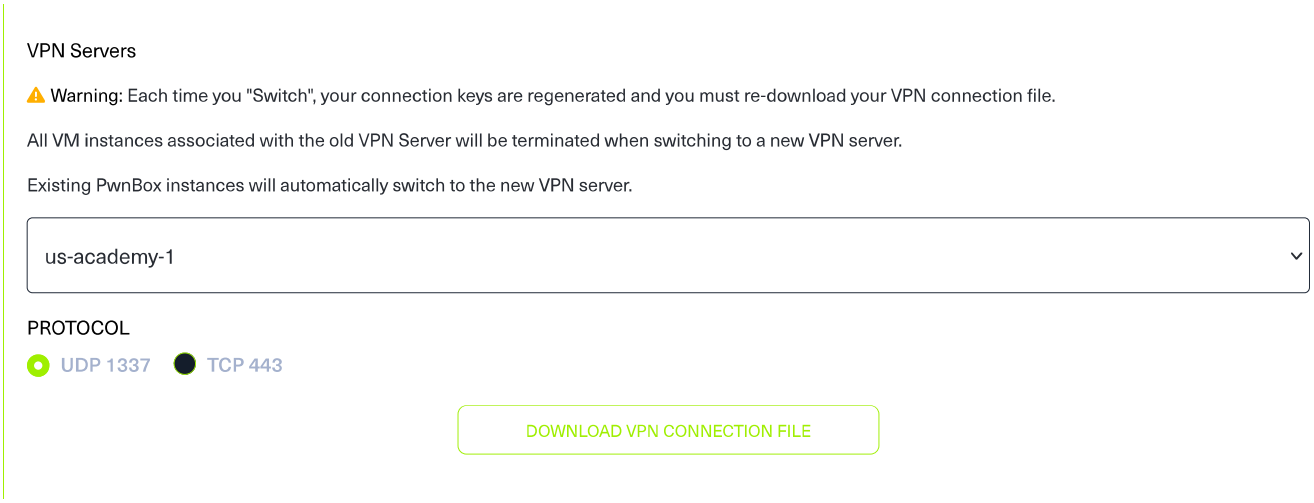
Integrated Terminal

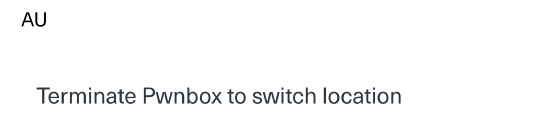
%ProgramFiles%	Equivalent of C:\Program Files. This location is where all the programs are installed on an x64 based system.
%ProgramFiles(x86)%	Equivalent of C:\Program Files (x86). This location is where all 32-bit programs running under WOW64 are installed. Note that this variable is only accessible on a 64-bit host. It can be used to indicate what kind of host we are interacting with. (x86 vs. x64 architecture)

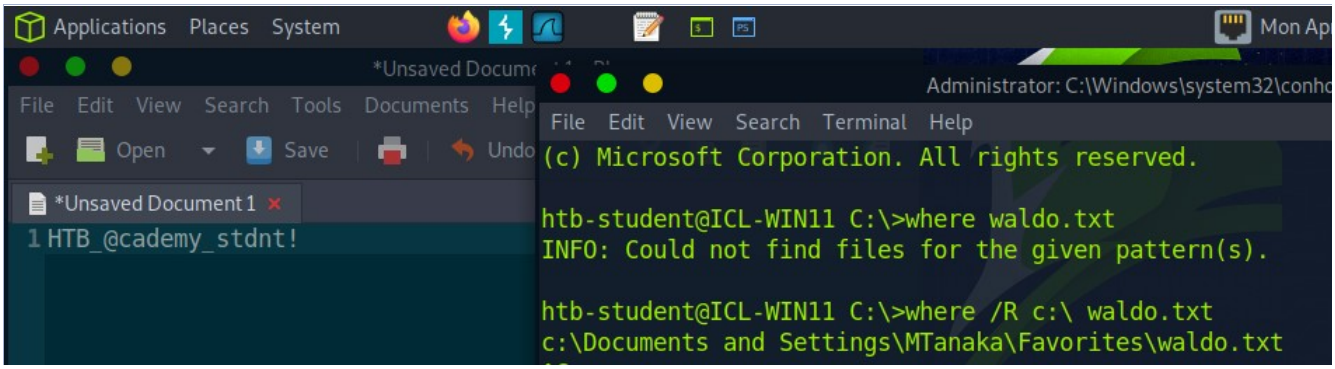
Provided here is only a tiny fraction of the information we can learn through enumerating the environment variables on a system. However, the abovementioned ones will often appear when performing enumeration on an engagement. For a complete list, we can visit the following link. Using this information as a guide, we can start gathering any required information from these variables to help us learn about our host and its target environment inside and out.

Moving On

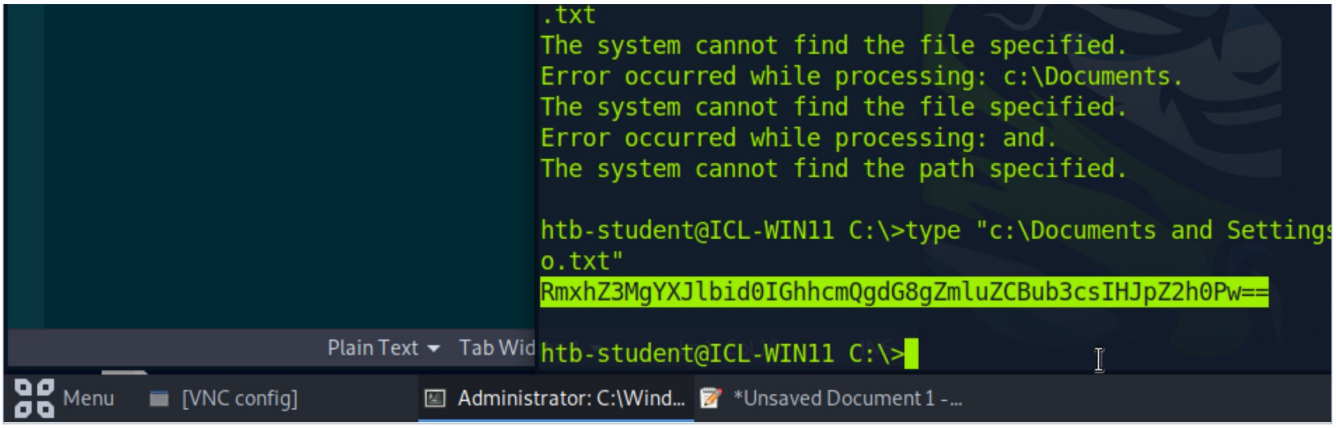
Following the end of this section, we should have a comfortable grasp of what environment variables are and how we can manage them in a system. Environment variables are a part of the core functionality of the Windows OS and are considered very useful to both attackers and defenders. Any modifications that can affect system-wide variables should be handled with extreme caution. If we find it necessary for our scripts or tools, make a new variable before editing one already on the system. Now that we have that information out of the way let us move on to using the command line to work with services on our host.







Integrated Terminal



Full Screen | Terminate | Reset

Connected to htb-byk9ife5jk:1 (htb-ac-819475)

Life Left: 86m +

Questions

Answer the question(s) below to complete this Section and earn cubes!

Cheat Sheet

Download VPN Connection File

Target: 10.129.203.105

Life Left: 110 minute(s) + Terminate

+ 0 What variable scope allows for universal access?

Submit your answer here...

Submit Hint

Previous Next

Cheat Sheet

Resources

Go to Questions

Table of Contents

- Working With Scheduled Tasks

PowerShell

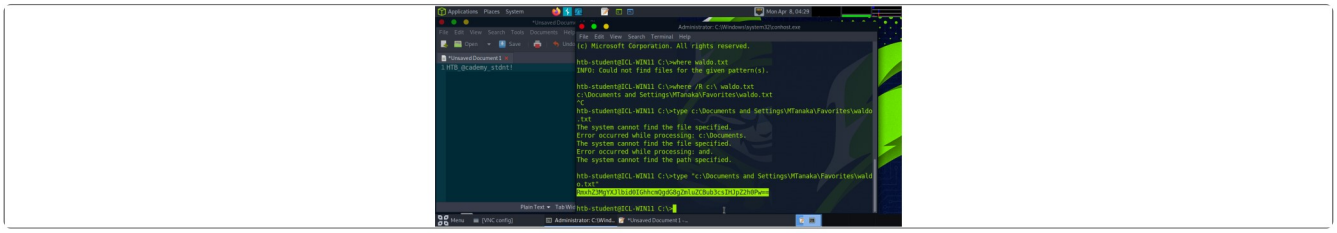
- CMD Vs. PowerShell
- All About Cmdlets and Modules
- User and Group Management
- Working with Files and Directories
- Finding & Filtering Content
- Working with Services
- Working with the Registry
- Working with the Windows Event Log
- Networking Management from The CLI
- Interacting with The Web
- PowerShell Scripting and Automation

Finish Strong

- Skills Assessment

Beyond This Module

My Workstation



Interact | Terminate | Reset | Life Left: 86m | +

Integrated Terminal